Architecture des microprocesseur

Module SLEEL602

Cours de deuxième semestre

L3 Electronique

Pré-requis : module SPUE42

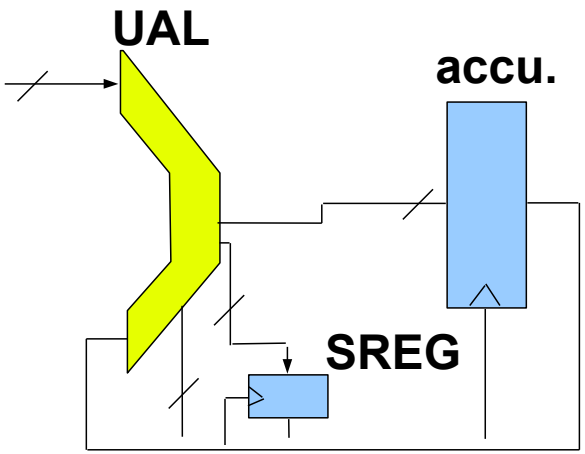
François VERDIER

Laboratoire LEAT

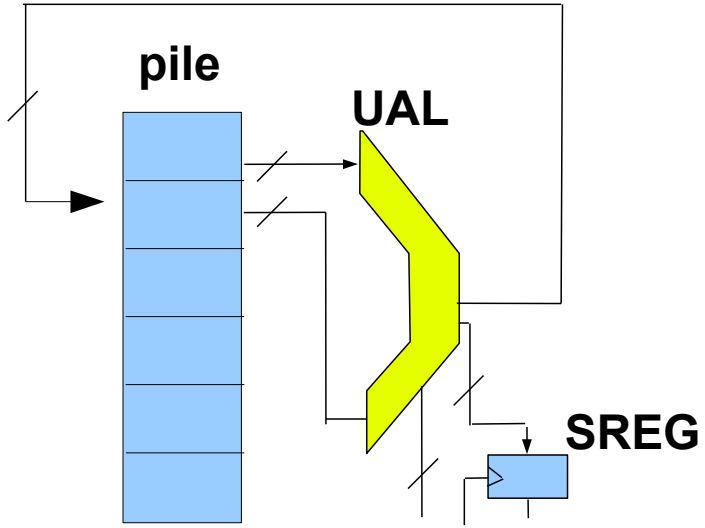
Université Côte d'Azur

Chapitre 3 :
Les branchements conditionnels

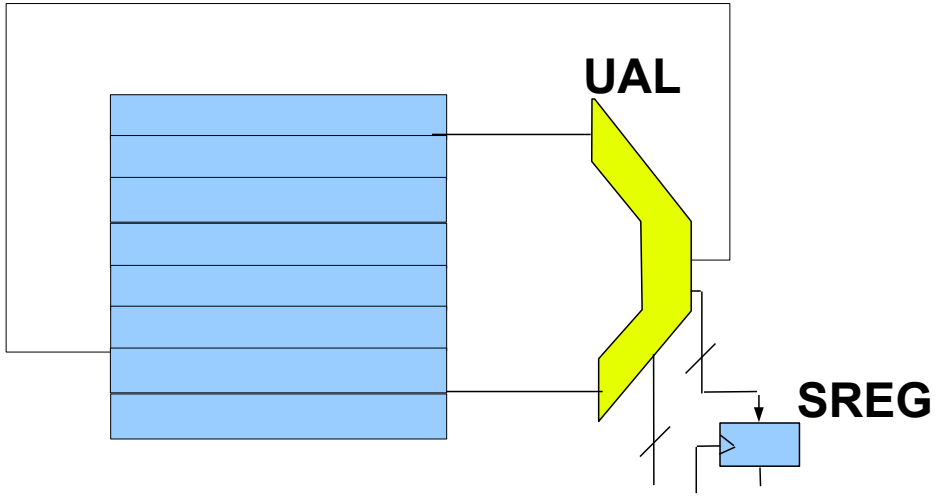
Le registre d'état des processeurs



Machine à accumulateur

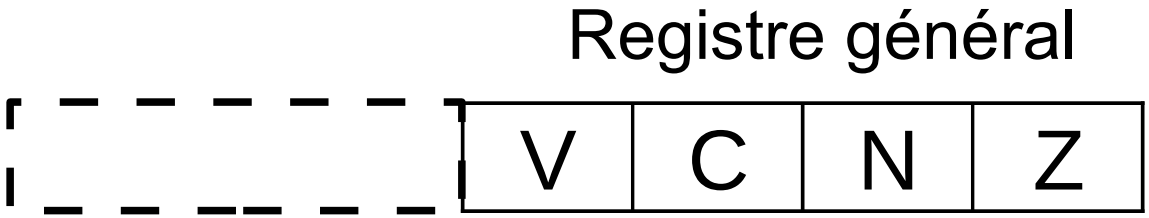


Machine à pile



Machine à registres généraux

Le registre SREG



- Z: (Zero) passe à 1 si le résultat de l'opération est égal à 0, passe à 0 sinon

- N: (Negate) passe à 1 si le résultat est négatif, passe à 0 sinon

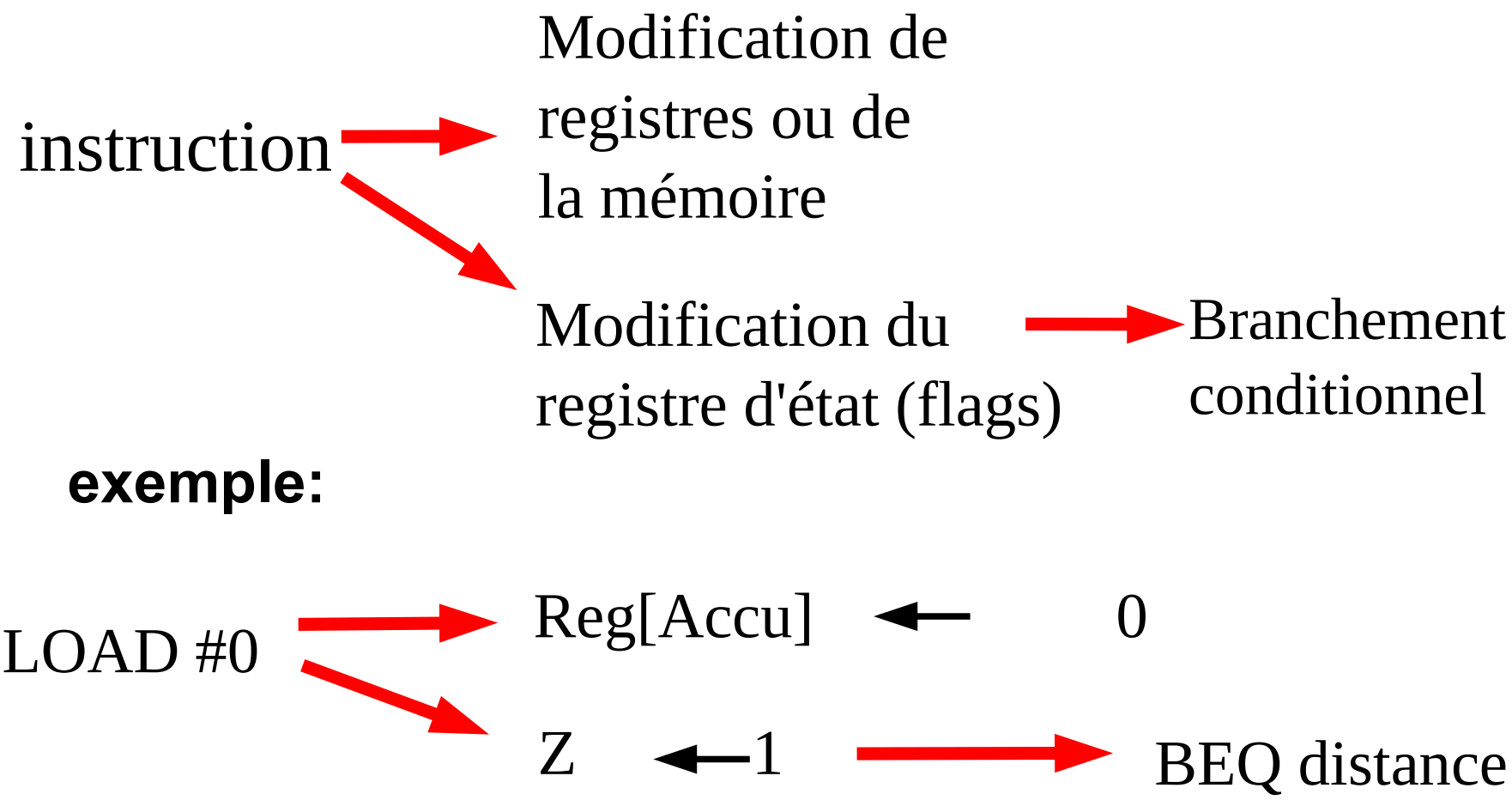
- C: (Carry) retenue déclenchée

- V: (Overflow) dépassement de capacité

- autres indicateurs...

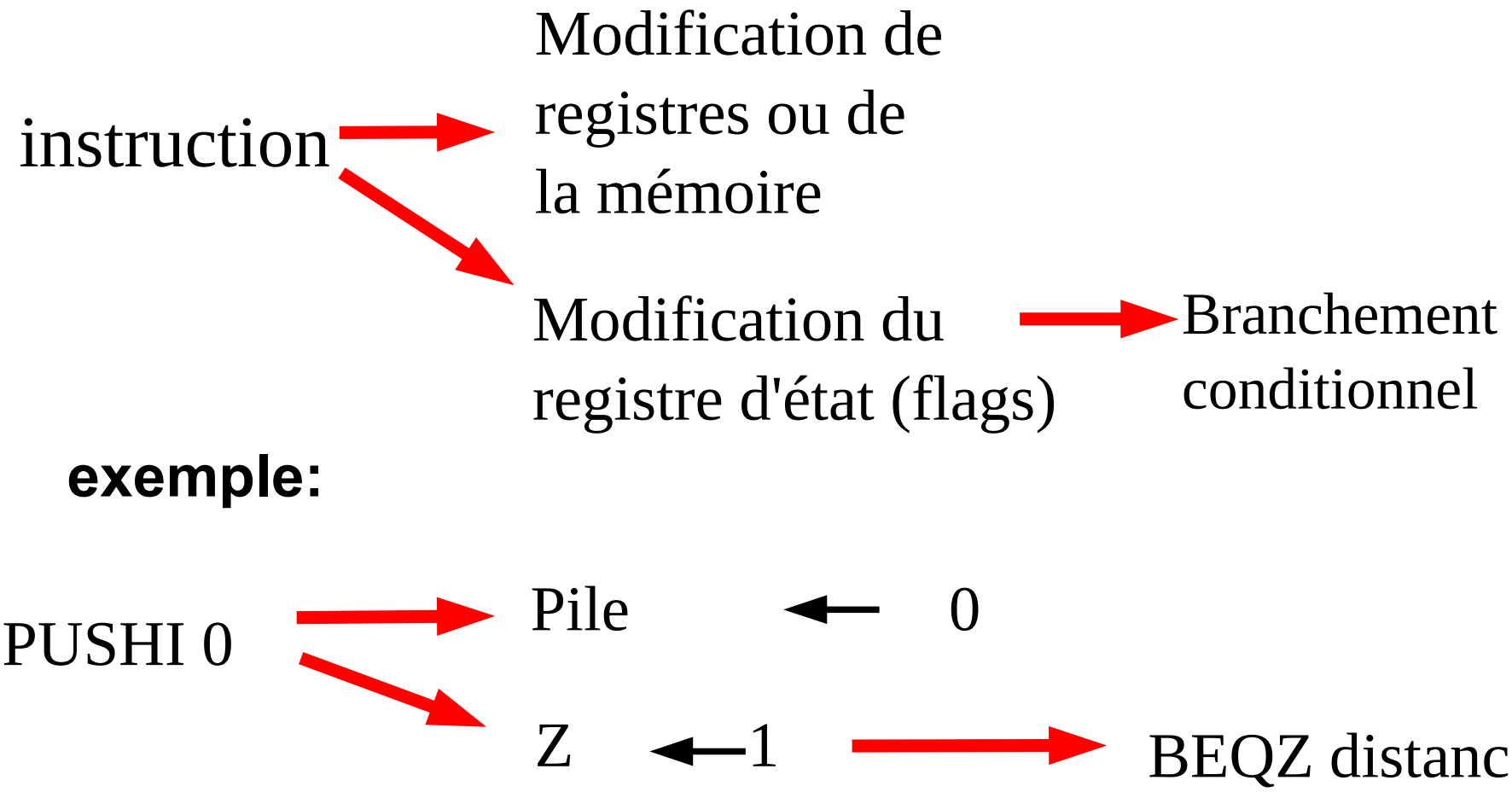
Comportement

machine à accumulateur



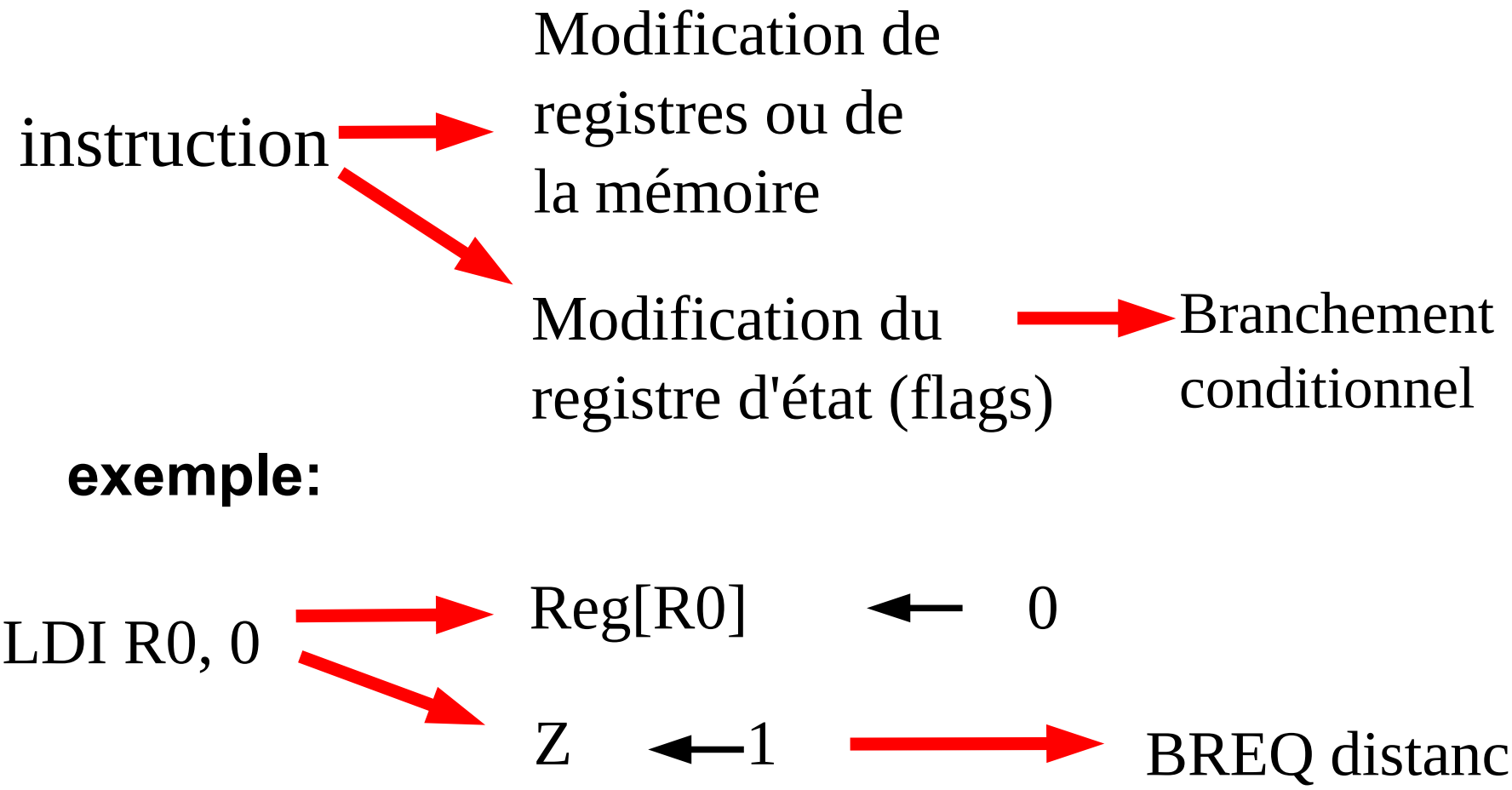
Comportement

machine à pile



Comportement

machine à registres généraux



Comportement

machine à accumulateur

```
        LOAD  #0
        BEQ   test
        LOAD  #1
test:   ADDI  1
```

Réellement, c'est la distance à parcourir en cas de test reussi

Instruction CMP

```
        LOAD #0
        CMPI 1
        BNE  test
        LOAD #1
test:   ADDI 1
```

accumulateur et valeur

Instructions de branchement

machine à accumulateur

inst.	opération	opcode	Inh	Imm	Dir	InDir
BEQ	*si Z alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	10100XX		/3		
BNE	*si Z alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	10101XX		/3		
BLT	*si N alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	10110XX		/3		
BLE	*si N + Z alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	10111XX		/3		
BGE	*si N alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	11000XX		/3		
BGT	*si N + Z alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	11001XX		/3		
BC	*si C alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	11010XX		/3		
BNC	*si C alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	11011XX		/3		
BV	*si V alors* $Reg[PC] \leftarrow Reg[PC] + depl$ *sinon* $Reg[PC] \leftarrow Reg[PC] + 3$	11100XX		/3		
BNV	*si V alors* $Reg[PC] \leftarrow Reg[PC] + 3$ *sinon* $Reg[PC] \leftarrow Reg[PC] + depl$	11101XX		/3		
B	$Reg[PC] \leftarrow Reg[PC] + depl$	11110XX		/3		
NOP	*aucune operation*	11111XX	/1			

Comportement

machine à pile

```
        PUSHI 0
        BEQZ test
        PUSH I 1
test:   PUSHI 1
        ADD
```

Réellement, c'est la distance à parcourir en cas de test reussi

Instruction CMP

```
        PUSHI 0
        PUSHI 1
        CMP
        BNEZ test
        PUSHI 2
test:   ADD
```

Instructions de branchement

machine à pile

Control Transfer

instruction	operation
beqz	Reg[pc] <-- if Z then d else Reg[pc]+1
bnez	Reg[pc] <-- if Z then Reg[pc]+1 else d
bltz	Reg[pc] <-- if N then d else Reg[pc]+1
bgez	Reg[pc] <-- if N then Reg[pc]+1 else d
bc	Reg[pc] <-- if C then d else Reg[pc]+1
bnc	Reg[pc] <-- if C then Reg[pc]+1 else d
bt	Reg[pc] <-- d

Comportement

machine à registres généraux

```
        LDI   R16, 0
        BREQ  test
        LDI   R16, 1
test:   LDI   R17, 1
        ADD   R16, R17
```

Réellement, c'est la distance à parcourir en cas de test reussi

Instruction CPI

```
        LDI   R16, 0
        CPI   R16, 1
        BRNE  test
        LDI   R16, 1
test:   LDI   R17, 1
        ADD   R16, R17
```

Instructions de branchement

machine à registres généraux

Mnem.	Ops	Description	Opération	Flags	#Clocks
CPSE	Rd, Rr	Comp. and skip	Si Rd=Rr PC ← PC+2/3		1/2/3
CP	Rd, Rr	Compare	Rd - Rr	H,S,V,N,Z,C	1
CPC	Rd, Rr	Comp. w/ C	Rd - Rr - C	H,S,V,N,Z,C	1
CPI	Rd, K	Compare	Rd - K	H,S,V,N,Z,C	1
SBRC	Rd, b	Skip if not bit	Si Rd(b)=0 PC ← PC+2/3		1/2/3
SBRS	Rd, b	Skip if bit	Si Rd(b)=1 PC ← PC+2/3		1/2/3
SBIC	A, b	Skip if not I/O bit	Si IO(A,b)=0 PC ← PC+2/3		1/2/3
SBIS	A, b	Skip if I/O bit	Si IO(A,b)=1 PC ← PC+2/3		1/2/3
BRBS	s, K	Br. if Bit=1	Si SREG(s)=1 PC ← PC+K+1		1/2
BRBC	s, K	Br if Bit=0	Si SREG(s)=0 PC ← PC+K+1		1/2
BREQ	K	Br. if Equal	Si Z=1 PC ← PC+K+1		1/2
BRNE	K	Br. if not equal	Si Z=0 PC ← PC+K+1		1/2
BRCS	K	Br. if carry set	Si C=1 PC ← PC+K+1		1/2
BRCC	K	Br. if carry clear	Si C=0 PC ← PC+K+1		1/2
BRSH	K	Br. if half carry	Si H=1 PC ← PC+K+1		1/2
BRLO	K	Br. if lower	Si C=0 PC ← PC+K+1		1/2

Les structures de contrôle

Alternative

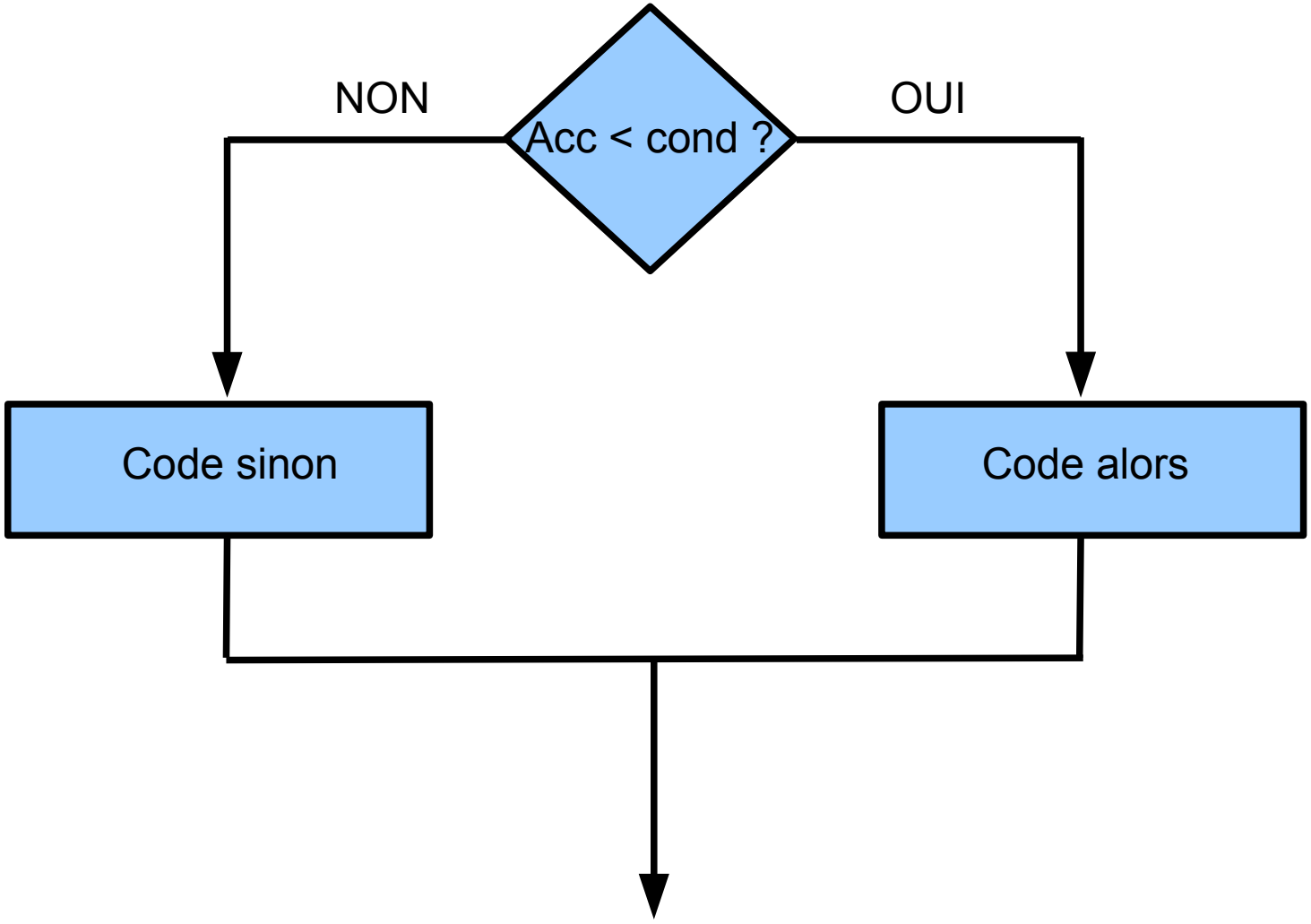
```
if (acc < const) {
        ...
        <code alors>
        ...
} else {
        ...
        <code sinon>
        ...
}
```

```
si:cmpi const
        bge sinon
        ...
        <code alors>
        ...
        b finsi
sinon:
        ...
        <code sinon>
        ...
finsi:
```

!!!

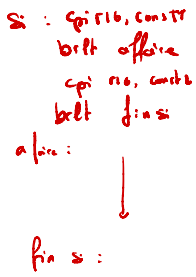
étiquettes

Alternative



Condition complexe: or

```
if (
   (acc < const1)
   ||
   (acc >= const2)
)
{
   ...
   <code>
   ...
}
```



```
si: cmpi const1
    blt afaire
    cmpi const2
    blt finsi
afaire:
    ...
        <code>
        ...
finsi:
```

R16 : 13_{10} = 0 0 0 0 1 1 0 1

Const2 : 15_{10} = 0 0 0 0 1 1 1 1

Const1 : 2_{10} : 0 0 0 0 0 0 1 0

```
si: cpi R16, 2
    brlt afara
    cpi R16, 15
    brlt fin si
afara: ldi R16, 0
fini si:
```

R16	Z	N
13	0	0
13	0	0
13	0	1
13	0	1



S: N = 1.

Condition complexe: and

```
if (
   (acc < const1)
   &&
   (acc >= const2)
)
{
   ...
   <code>
   ...
}
```

```
si: cmpi const1
        bge  finsi
        cmp const2
        blt  finsi
        ...
        <code>
        ...
finsi:
```

R16 : 13_{10} = 0 0 0 0 1 1 0 1

Const2 : 10 = 0 0 0 0 1 0 1 0

Const1 : 18 : 0 0 0 1 0 0 1 0

```
si:   cpi  R16, 18
      brge fin si
      cpi  R16, 10
      brlt fin si
      ldi  R16, 0
fin si
```

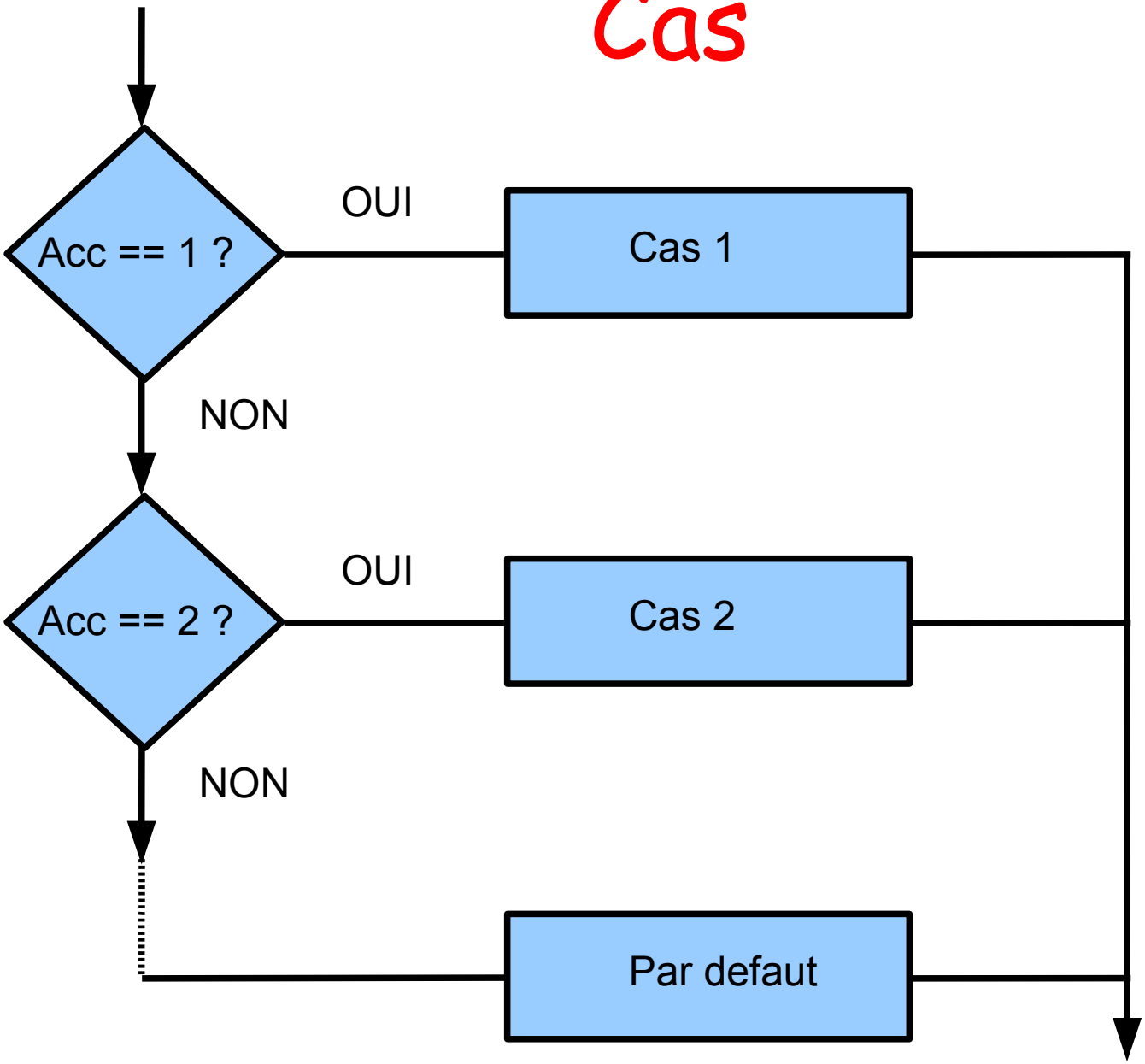
R16	Z	N
13	0	1
13	0	1
13	0	0
13	0	0
0	1	0

Cas

```
switch (acc) {
   case 1:     …
               …
               break;
   case 2:     …
               …
               break;
   …
   default: …
}
```

```
switch:
cas1:  cmpi   1
               bne    case2
               …
               …
               b  finsw
cas2:  cmpi   2
               bne    case3
               …
               b  finsw
pardefaut:     …
finsw:
```

Cas

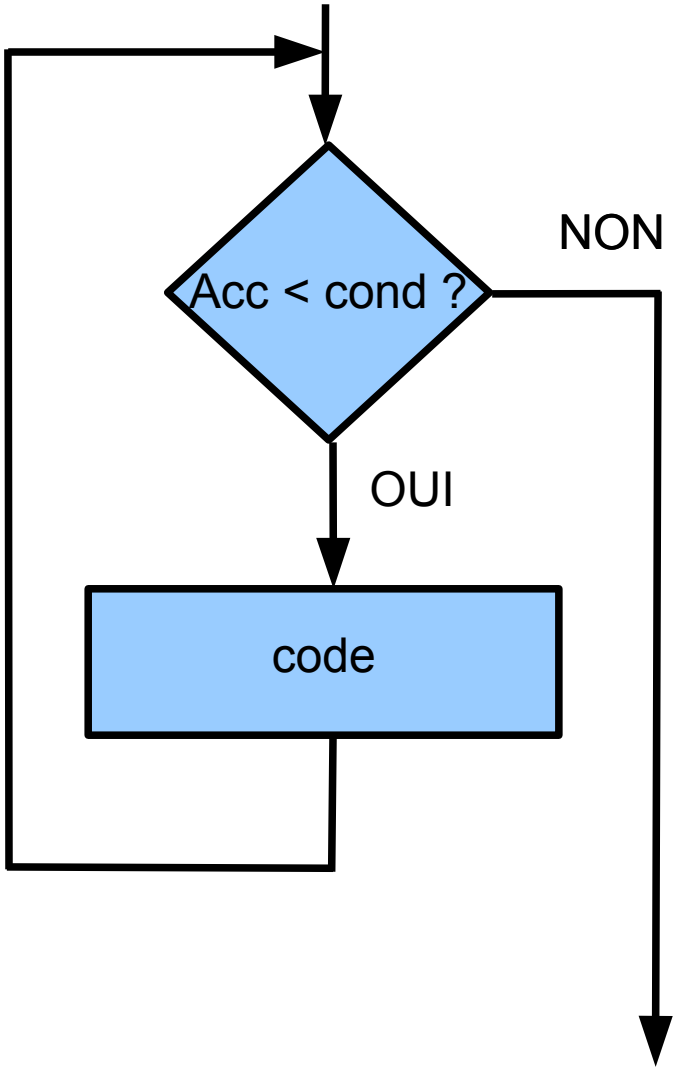


Tant que

```
While ( acc < const ) {
   ...
   <code>
   ...
}
```

```
tantque:
        cmpi  const
        bge fintantque
        ...
        <code>
        ...
        b   tantque
fintantque:
```

Tant que

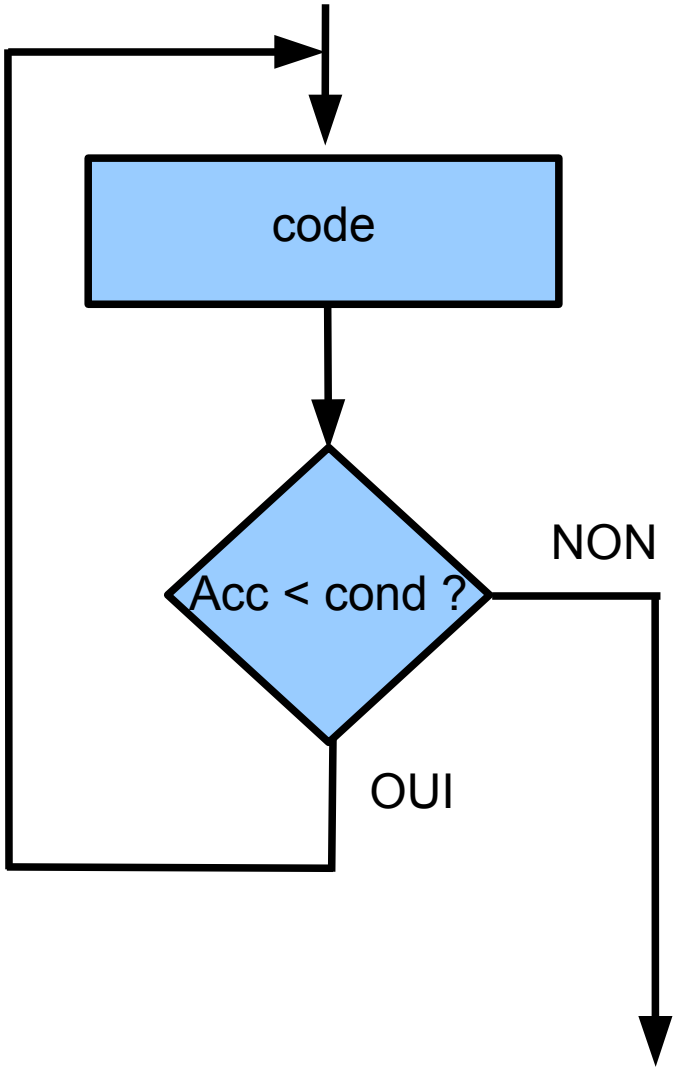


Repete jusqu'a ou faire tantque

```
do {
    ...
    <code>
    ...
} while (acc < const)
```

```
faire:
    ...
    <code>
    ...
    cmpi const
    blt faire
```

Repete jusqu'à ou faire tant que



Exemples de programmes

Application1 : pgcd

Algorithme: Convergence par soustraction

// entrées: A et B: entiers positifs

// sortie: pgcd(A,B): entier positif

tant que A ≠ B

si A > B alors

A ← A - B

sinon

B ← B - A

fin si

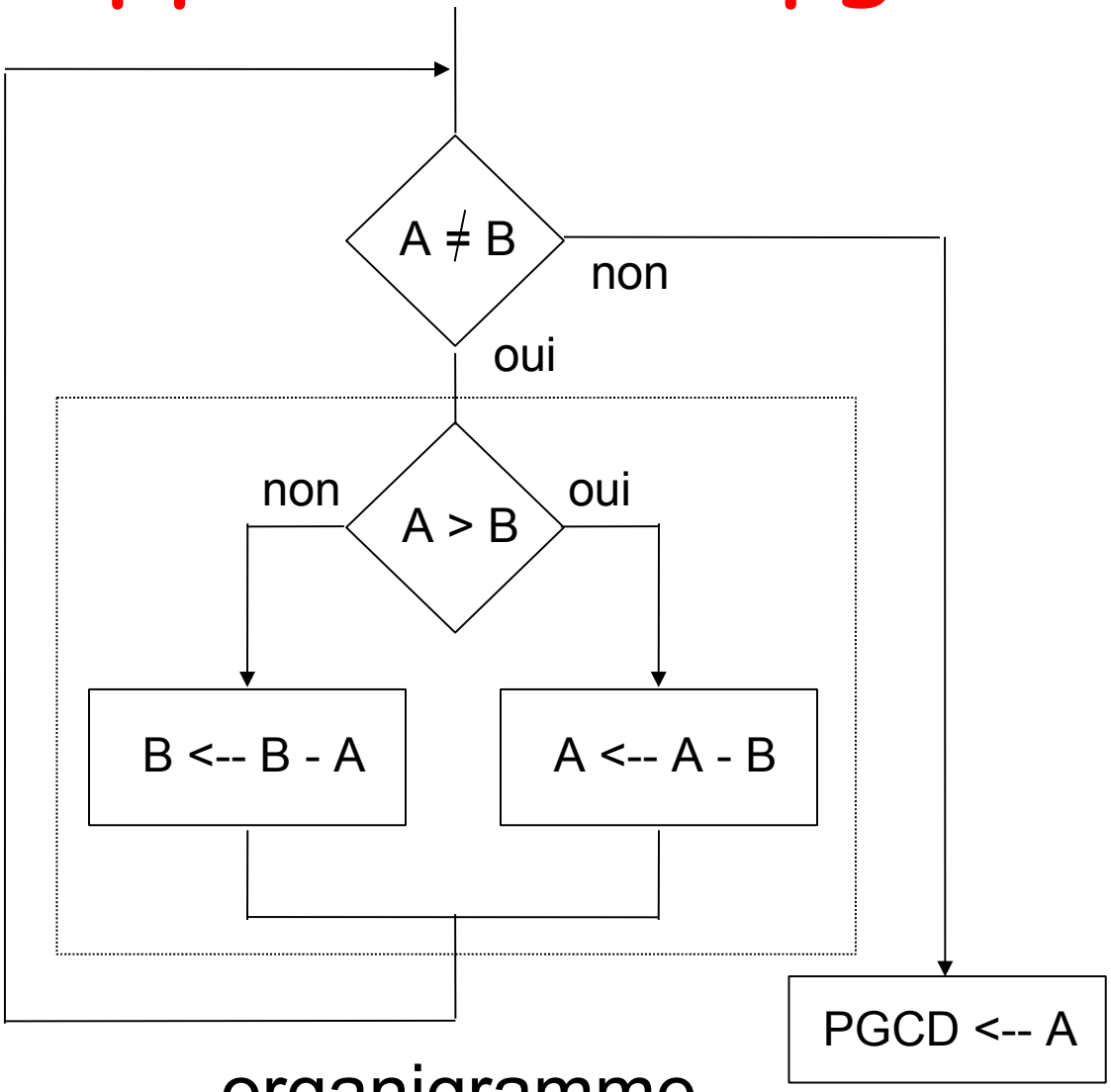
fin tant que

pgcd ← A

Invariant de boucle:
A et B entiers positifs

Variation:
|A-B| strictement décroissant

Application1 : pgcd



organigramme

pgcd: implementation par machine à pile

```
        PUSH  A       ; | A
        PUSH  B       ; | A B
Bcl:    CMP           ; pile inchangée et test de A-B
        BEQZ  Done:   ; si Z=1 alors brancher à Done
        BLTZ  Else:   ; si A est inférieur à B aller à Else
        SWAP          ; | B A
        OVER          ; | B A B
        SUB           ; | B A-B
        BT    Bcl:        ; on recommence à Bcl
Else:   OVER          ; | A B A
        SUB           ; | A B-A
        BT    Bcl:    ; on recommence à Bcl
Done:   DROP          ; | A
        POP   PGCD    ; PGCD = A
```

pgcd: implementation par machine à accumulateur

```
        LOAD    A           ; Accu = A
        STORE   sauvA       ; mem[sauvA] = A
        LOAD    B           ; Accu = B
        STORE   sauvB       ; mem[sauvB] = B
Bcl:    SUB     sauvA       ; Accu = B-A
        BEQ     Done:       ; si Z=1 aller à Done
        BLT     Then:       ; si N=1 aller à Else
        STORE   sauvB       ; sinon, mem[sauvB] = B-A
        B       Bcl:
Then:   NEG                 ; Accu = A-B
        STORE   sauvA       ; mem[sauvA] = A-B
        LOAD    sauvB       ; Accu = B
        B       Bcl:
Done:   LOAD    sauvA       ; Accu = A
        STORE   PGCD        ; mem[PGCD] = A
```

Application 2:

Somme des N premiers nombres impairs

Definition: $$S = \sum_{k=0}^{k=N-1} (2*k+1)$$

Algorithme

```
// entrées: N: entier positif
// sortie: S: entier positif
odd:=1:integer;    S = 0; // init
faire N fois
   S = S + odd;
   odd = odd + 2
fait
```

Addition simple grâce à l'utilisation d'une variable locale

Implementation (pile)

```
                        ; initialisations
pushi    1              ; |odd
pushi    0              ; |odd S
push     100            ; |odd S N
                        ; loop
pushi    1              ; |odd S N 1
sub                     ; |odd S N-1=N'
bltz     Fin:           ; branchement si N'<0
roll                    ; |N' odd S
over                    ; |N' odd S odd
add                     ; |N' odd S+odd=S'
roll                    ; |S' N' odd
```

Cette structure pourrait être un invariant de boucle

Nouvelle valeur de N

Nouvelle valeur de S

Implementation (pile)

```
 pushi   2          ; |S' N' odd 2
 add                ; |S' N' odd+2=odd'
 roll               ; |odd' S' N'
 bt     -16         ;
Fin :
 drop               ; |odd S
 pop     101        ; |odd
 drop               ; |
```

Nouvelle valeur de odd

invariant de la boucle

Conseils à propos de la machine à pile

- Moins il y aura de références mémoires, mieux cela vaudra
- Utilisez dup, over pour sauver des variables temporaires sur la pile
- Utilisez swap, roll pour réorganiser les variables sur la pile
- Utilisez drop, pop pour effacer les variables devenues inutiles
- Trouver l'ordre des variables qui fait ressortir l'invariant de boucle

Implementation (mac)

```
Debut: loadi 0
       store 102
       Load 102
       addi 1
       store 102
       cmpi 100
       beq Fin
       Load 100
       Add 101
       store 100
       load 101
       addi 2
       store 101
       bt Debut

Fin:   bt Fin
```